

Deployment of Swiggy Application

Name: Pooja Prakash Belgaumkar

1. Real-Time DevOps Project: Deployment of Swiggy Clone Application Using Modern DevOps Tools and Cloud Services

- This project demonstrates end-to-end application deployment using Infrastructure as Code (Terraform), CI/CD automation (Jenkins), Code Quality Analysis (SonarQube), Security Scanning (OWASP Dependency Check & Trivy), and Containerization (Docker) on AWS Cloud.
- Terraform is used to provision and manage the infrastructure required for application deployment.
- In **main.tf**, all necessary AWS resources such as EC2 instance and Security Groups are defined for deployment.
- In **provider.tf**, the AWS provider configuration and working region are specified.
- In **resource.sh**, required tools such as Docker, Jenkins, Node.js, and Trivy are installed automatically when the EC2 instance is launched using user data.

```
Terminal  Help  <  >  Q Swiggy App Project

Main.tf  X  provider.tf  $ resource.sh

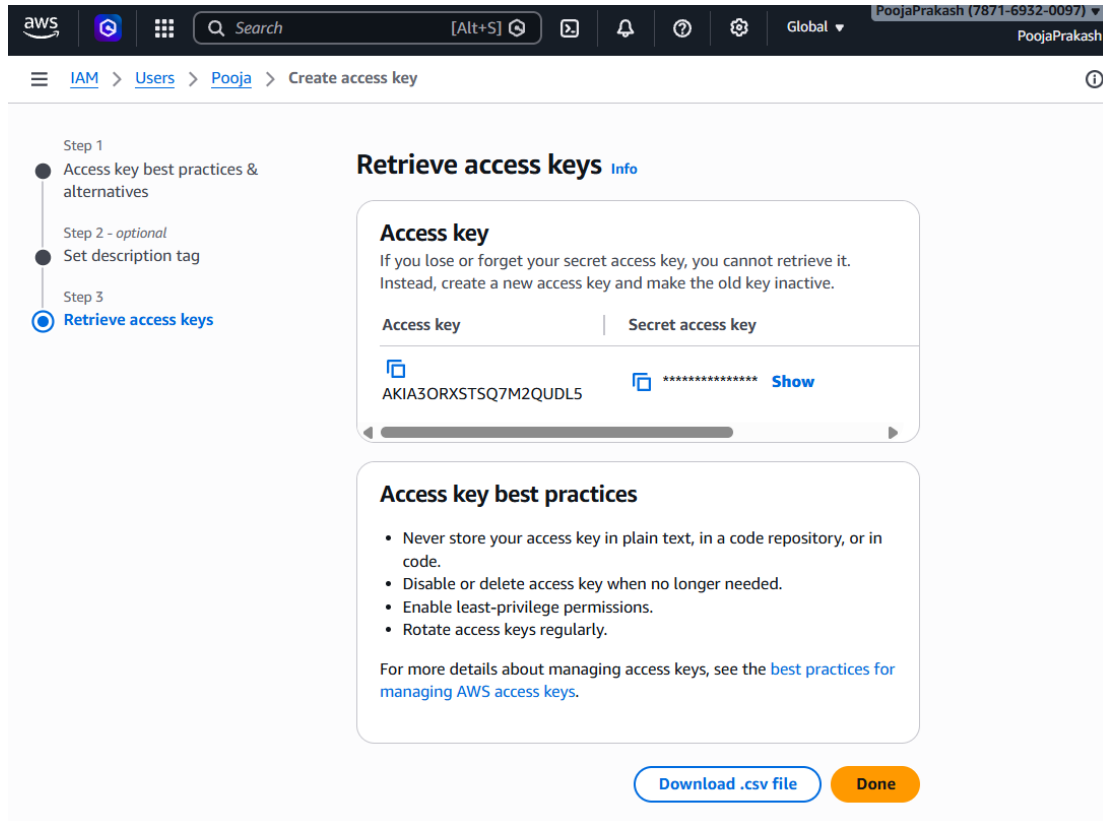
Main.tf > ...
1  # Security Group
2  resource "aws_security_group" "Project-SG" {
3      name           = "Project-SG"
4      description    = "Open 22,443,80,8080,9000"
5
6      # Define a single ingress rule to allow traffic on all specified ports
7      ingress = [
8          for port in [22, 80, 443, 8080, 9000, 3000] : {
9              description    = "TLS from VPC"
10             from_port      = port
11             to_port        = port
12             protocol        = "tcp"
13             cidr_blocks     = ["0.0.0.0/0"]
14             ipv6_cidr_blocks = []
15             prefix_list_ids = []
16             security_groups = []
17             self            = false
18         }
19     ]
20
21     egress {
22         from_port = 0
23         to_port   = 0
24         protocol  = "-1"
25         cidr_blocks = ["0.0.0.0/0"]
26     }
27
28     tags = {
29         Name = "Project-SG"
30     }
31 }
32
```

```
33
34  # EC2 Instance
35  resource "aws_instance" "web" {
36      ami           = "ami-073130f74f5ffb161"
37      instance_type = "m7i-flex.large"
38      key_name      = "linux1"
39      subnet_id     = "subnet-0cbc6ee27b50c1eda"
40      vpc_security_group_ids = [aws_security_group.Project-SG.id]
41      user_data      = templatefile("./resource.sh", {})
42
43      tags = {
44          Name = "Swiggy-App"
45      }
46      root_block_device {
47          volume_size = 30
48      }
49  }
```

```
Main.tf provider.tf X resource.sh
provider.tf > provider "aws"
1 terraform {
2     required_providers {
3         aws = {
4             source = "hashicorp/aws"
5             version = "~> 5.0"
6         }
7     }
8 }
9
10 # Configure the AWS Provider
11 provider "aws" {
12     region = "eu-north-1"
13 }
```

```
Main.tf provider.tf X resource.sh
$ resource.sh
1 #!/bin/bash
2 sudo apt update -y
3 wget -O - https://packages.adoptium.net/artifactory/api/gpg/key/pub
4 echo "deb [signed-by=/etc/apt/keyrings/adoptium.asc] https://package
5 sudo apt update -y
6 sudo apt install temurin-17-jdk -y
7 /usr/bin/java --version
8 curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
9 echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://
10 sudo apt-get update -y
11 sudo apt-get install jenkins -y
12 sudo systemctl start jenkins
13 sudo systemctl status jenkins
14
15 #install docker
16 sudo apt-get update
17 sudo apt-get install docker.io -y
18 sudo usermod -aG docker ubuntu
19 newgrp docker
20 sudo chmod 777 /var/run/docker.sock
21 docker run -d --name sonar -p 9000:9000 sonarqube:lts-community
22
23 # install trivy
24 sudo apt-get install wget apt-transport-https gnupg lsb-release -y
25 wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key
26 echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] https://aquasec
27 sudo apt-get update
28 sudo apt-get install trivy -y
```

- An IAM user is created in the AWS Console to generate Access Key and Secret Key for configuring AWS CLI in VS Code to work with Terraform securely.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\POOJA\Desktop\DevOps Study Materials\Github Portfolio\Projects\Swiggy App Project\Terraform-Script-Swiggy> aws configure
AWS Access Key ID [*****UDL5]: AKIA3ORXSTSQ7M2QUDL5
AWS Secret Access Key [*****BY/E]: UqEG+6UaDiYBfErich7MC06Qdt+AiN3Xo/iBY/E
Default region name [eu-north-1]: eu-north-1
Default output format [json]: json
PS C:\Users\POOJA\Desktop\DevOps Study Materials\Github Portfolio\Projects\Swiggy App Project\Terraform-Script-Swiggy> |
```

- The command *terraform init* is executed to initialize Terraform and download the required provider plugins.

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Swiggy App Project\Terraform-Script-Swiggy> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Swiggy App Project\Terraform-Script-Swiggy>
```

- The command *terraform plan* is used to preview the infrastructure changes that will be applied.

```
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Swiggy App Project\Terraform-Script-Swiggy> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami              = "ami-001f2488b35ca8aad"
  + arn              = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone = (known after apply)
  + cpu_core_count   = (known after apply)
}
```

- The command *terraform apply* is executed to provision all resources defined in the Terraform configuration files.

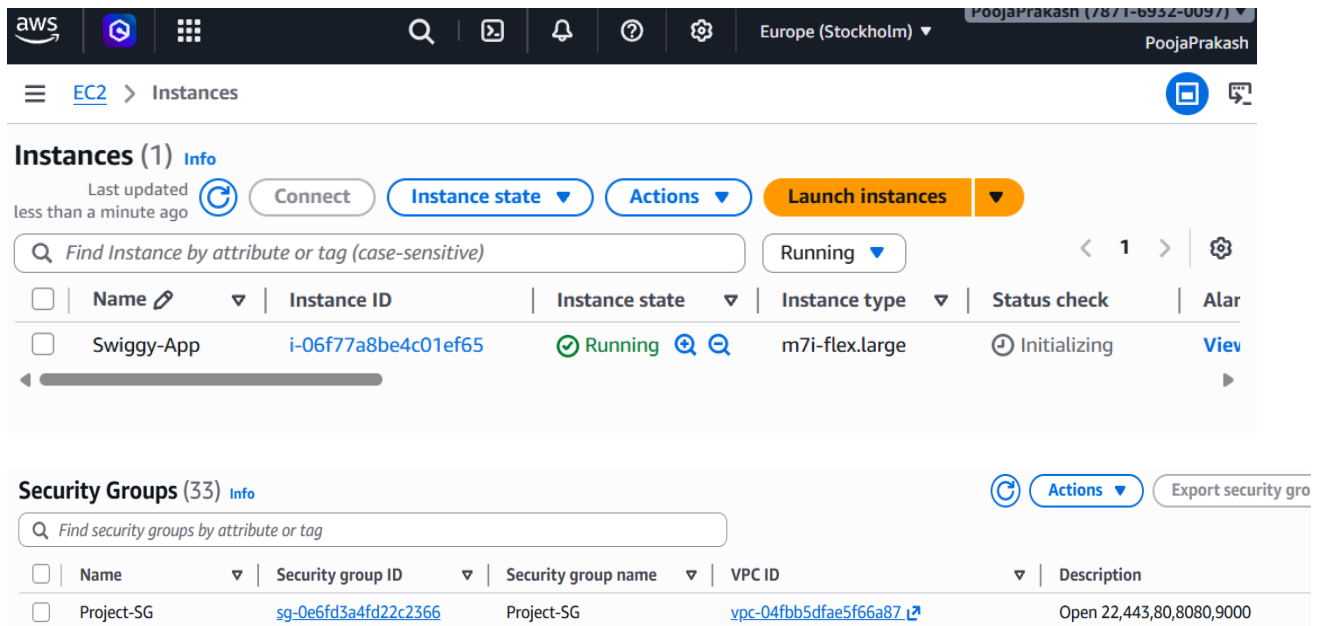
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Swiggy App Project\Terraform-Script-Swiggy> terraform apply --auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_security_group.Project-SG: Creating...
aws_security_group.Project-SG: Creation complete after 7s [id=sg-0e6fd3a4fd22c2366]
aws_instance.web: Creating...
aws_instance.web: Still creating... [00m10s elapsed]
aws_instance.web: Creation complete after 19s [id=i-06f77a8be4c01ef65]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
PS C:\Users\P00JA\Desktop\DevOps Study Materials\GitHub Portfolio\Projects\Swiggy App Project\Terraform-Script-Swiggy>
```

- The created resources are verified in the AWS Console after successful deployment.



The screenshot shows the AWS Management Console interface. At the top, the navigation bar includes the AWS logo, a search icon, and the user's name 'PoojaPrakash' with their email '7871-6952-0097'. The main content area is divided into two sections.

Instances (1) Info

Last updated less than a minute ago. Buttons: Connect, Instance state, Actions, Launch instances.

Find Instance by attribute or tag (case-sensitive) Running 1

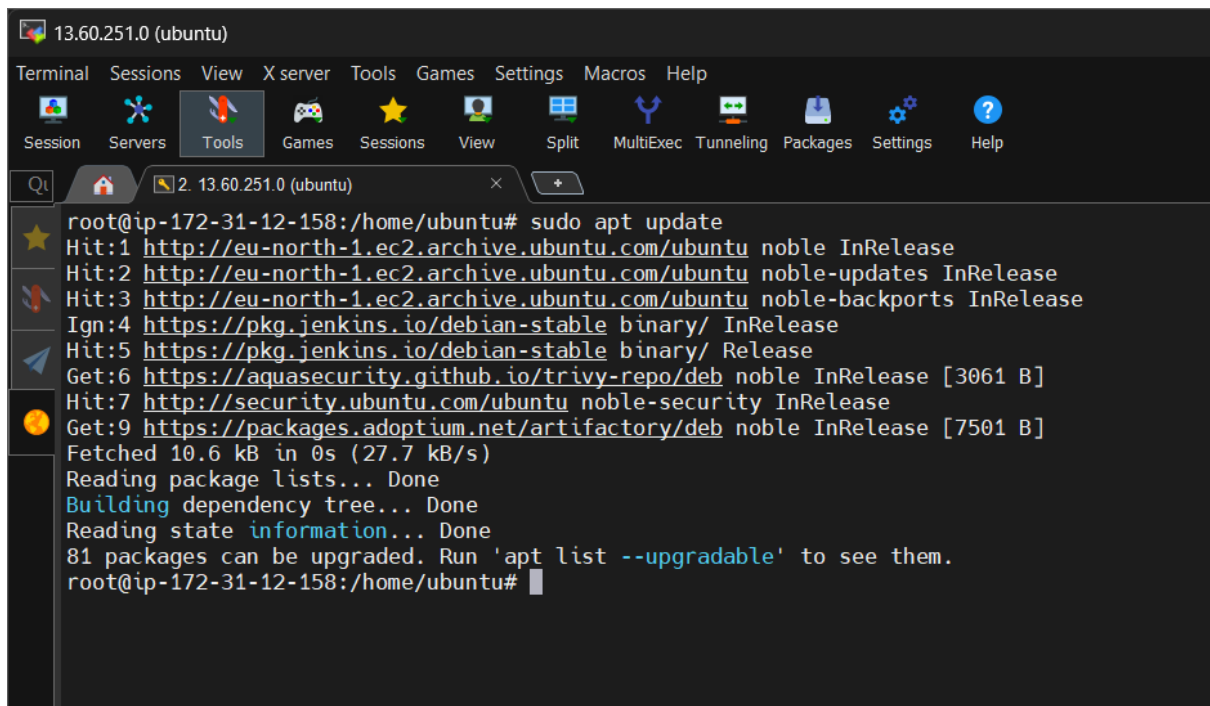
	Name	Instance ID	Instance state	Instance type	Status check	Alarm
☐	Swiggy-App	i-06f77a8be4c01ef65	Running	m7i-flex.large	Initializing	View

Security Groups (33) Info

Find security groups by attribute or tag

	Name	Security group ID	Security group name	VPC ID	Description
☐	Project-SG	sg-0e6fd3a4fd22c2366	Project-SG	vpc-04fbb5dfae5f66a87	Open 22,443,80,8080,9000

- The EC2 instance launched by Terraform is accessed via SSH, and the system is updated.



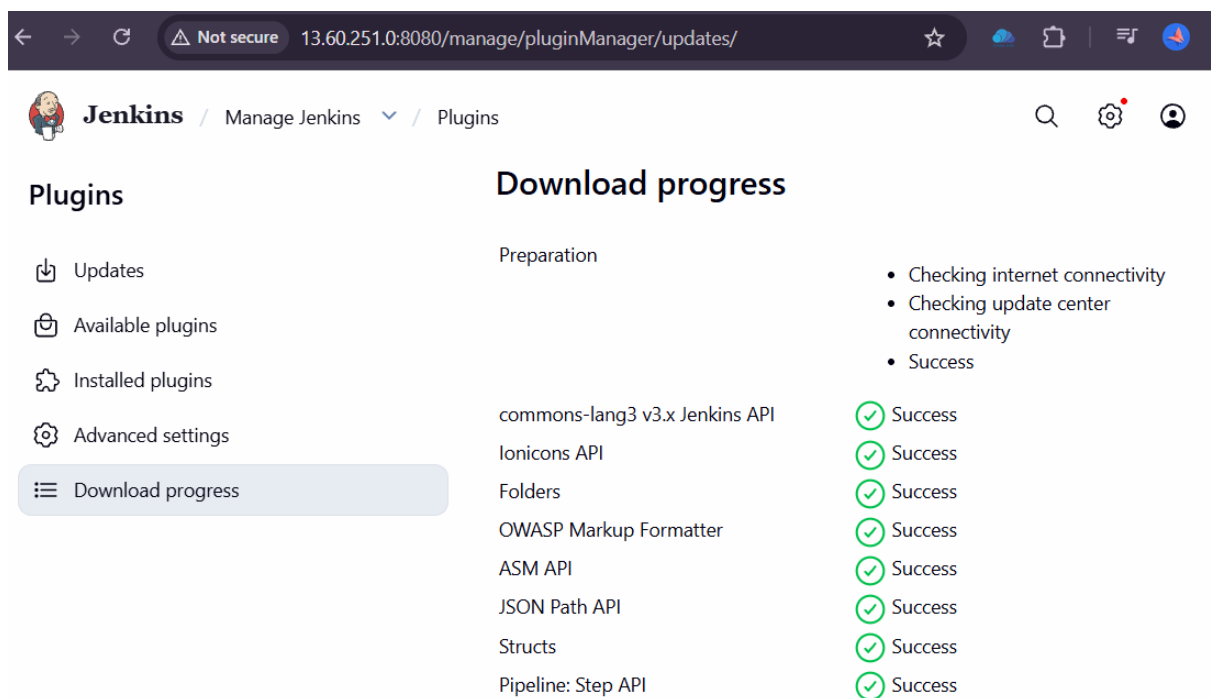
```

13.60.251.0 (ubuntu)
Terminal Sessions View X server Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help
2. 13.60.251.0 (ubuntu)
root@ip-172-31-12-158:/home/ubuntu# sudo apt update
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Ign:4 https://pkg.jenkins.io/debian-stable binary/ InRelease
Hit:5 https://pkg.jenkins.io/debian-stable binary/ Release
Get:6 https://aquasecurity.github.io/trivy-repo/deb noble InRelease [3061 B]
Hit:7 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:9 https://packages.adoptium.net/artifactory/deb noble InRelease [7501 B]
Fetched 10.6 kB in 0s (27.7 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
81 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@ip-172-31-12-158:/home/ubuntu#
  
```

- All required tools installed through the **resource.sh** script are verified on the EC2 instance.

```
root@ip-172-31-12-158:/home/ubuntu# java --version
openjdk 21.0.10 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
root@ip-172-31-12-158:/home/ubuntu# docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
root@ip-172-31-12-158:/home/ubuntu# trivy --version
Version: 0.69.1
root@ip-172-31-12-158:/home/ubuntu# sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-02-22 14:29:03 UTC; 5min ago
     Main PID: 9866 (java)
        Tasks: 43 (limit: 9283)
      Memory: 648.9M (peak: 686.5M)
         CPU: 14.850s
      CGroup: /system.slice/jenkins.service
              └─9866 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/
```

- Jenkins is accessed through port 8080, and the necessary plugins are installed.



The screenshot displays the Jenkins web interface in a browser. The address bar shows the URL `13.60.251.0:8080/manage/pluginManager/updates/`. The page title is "Jenkins" and the navigation bar includes "Manage Jenkins" and "Plugins". The main content area is titled "Download progress" and shows a list of plugins being installed, all with green checkmarks indicating success.

Plugin	Status
Preparation	• Checking internet connectivity • Checking update center connectivity • Success
commons-lang3 v3.x Jenkins API	Success
Ionicons API	Success
Folders	Success
OWASP Markup Formatter	Success
ASM API	Success
JSON Path API	Success
Structs	Success
Pipeline: Step API	Success

- Global tools such as JDK, SonarQube Scanner, NodeJS, Docker, and Trivy are configured in Jenkins.

JDK installations

+ Add JDK

⋮ JDK

Name

jdk17

☒ Install automatically ?

⋮ Install from adoptium.net ?

Version ?

jdk-17.0.11+9 ▾

+ Add Installer

SonarQube Scanner installations

+ Add SonarQube Scanner

⋮ SonarQube Scanner

Name

sonar-scanner

☒ Install automatically ?

⋮ Install from Maven Central

Version

SonarQube Scanner 8.0.1.6346

+ Add Installer

NodeJS installations

+ Add NodeJS

⋮ NodeJS

Name

node25

☒ Install automatically ?

⋮ Install from nodejs.org

Version

NodeJS 25.6.1

Docker installations

+ Add Docker

⋮ Docker

Name

docker

☒ Install automatically ?

⋮ Download from docker.com

Docker version ?

latest

+ Add Installer

Dependency-Check installations

+ Add Dependency-Check

⋮ Dependency-Check

Name

DP-Check

☒ Install automatically ?

⋮ Install from github.com

Version

dependency-check 12.2.0

+ Add Installer

- SonarQube is accessed via port 9000, and an authentication token is generated.
- The SonarQube token is added to Jenkins Credentials for secure integration.

Not secure 13.60.251.0:9000/admin/users

You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately. [Learn More](#)

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration A

Administration

Configuration ▾ Security ▾ Projects ▾ System Marketplace

Users [Create User](#)

Create and administer individual users.

	SCM Accounts	Last connection	Groups	Tokens
A Administrator admin		< 1 hour ago	sonar-administrators sonar-users	0

1 of 1 shown

Tokens of Administrator

Generate Tokens

Name

Expires in

30 days

Generate

!

New token "token" has been created. Make sure you copy it now, you won't be able to see it again!

Copy

squ_aa532c39012c57062dd01534721bd289437918c0

Name	Type	Project	Last use	Created	Expiration	
token	User		Never	February 22, 2026	March 24, 2026	Revoke

Done



Jenkins / Manage Jenkins / Credentials / System / Global



Global

+ Add Credentials

Credentials that should be available everywhere.



Sonar-token

Sonar-token - Sonar-token



- A webhook is configured in SonarQube using the Jenkins URL to automatically update Quality Gate results after each build.

sonarqube

ProjectsIssuesRulesQuality ProfilesQuality GatesAdministration

?

Q Search for projects...

A

Administration

Configuration

Security

Projects

System

Marketplace

Webhooks

Create

Webhooks are used to notify external services when a project analysis is done. An HTTP POST request including a JSON payload is sent to each of the provided URLs. Learn more in the [Webhooks documentation](#).

No webhook defined.

Create Webhook

All fields marked with * are required

Name *

jenkins

✓

URL *

http://13.60.251.0:8080/sonarqube-webhook/

✓

Server endpoint that will receive the webhook payload, for example:
"http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example:
"https://myLogin:myPassword@my_server/foo"

Secret

If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header.

Create

Cancel

- The SonarQube server is configured in Jenkins using the server URL and authentication token.

 **Jenkins** / Manage Jenkins / System

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as er

☐ Environment variables

SonarQube installations

List of SonarQube installations

Name

sonar-server

Server URL

Default is http://localhost:9000

http://13.60.251.0:9000

Server authentication token

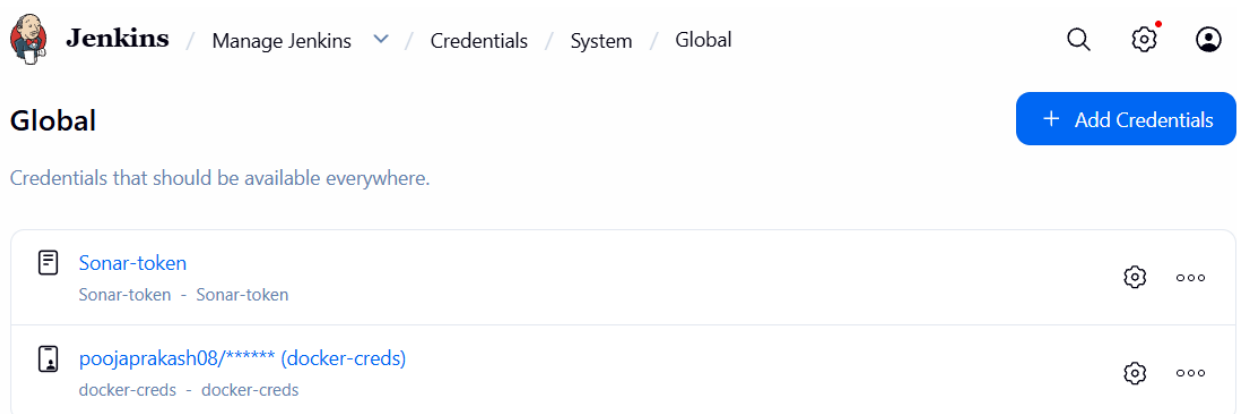
SonarQube authentication token. Mandatory when anonymous access is disabled.

Sonar-token

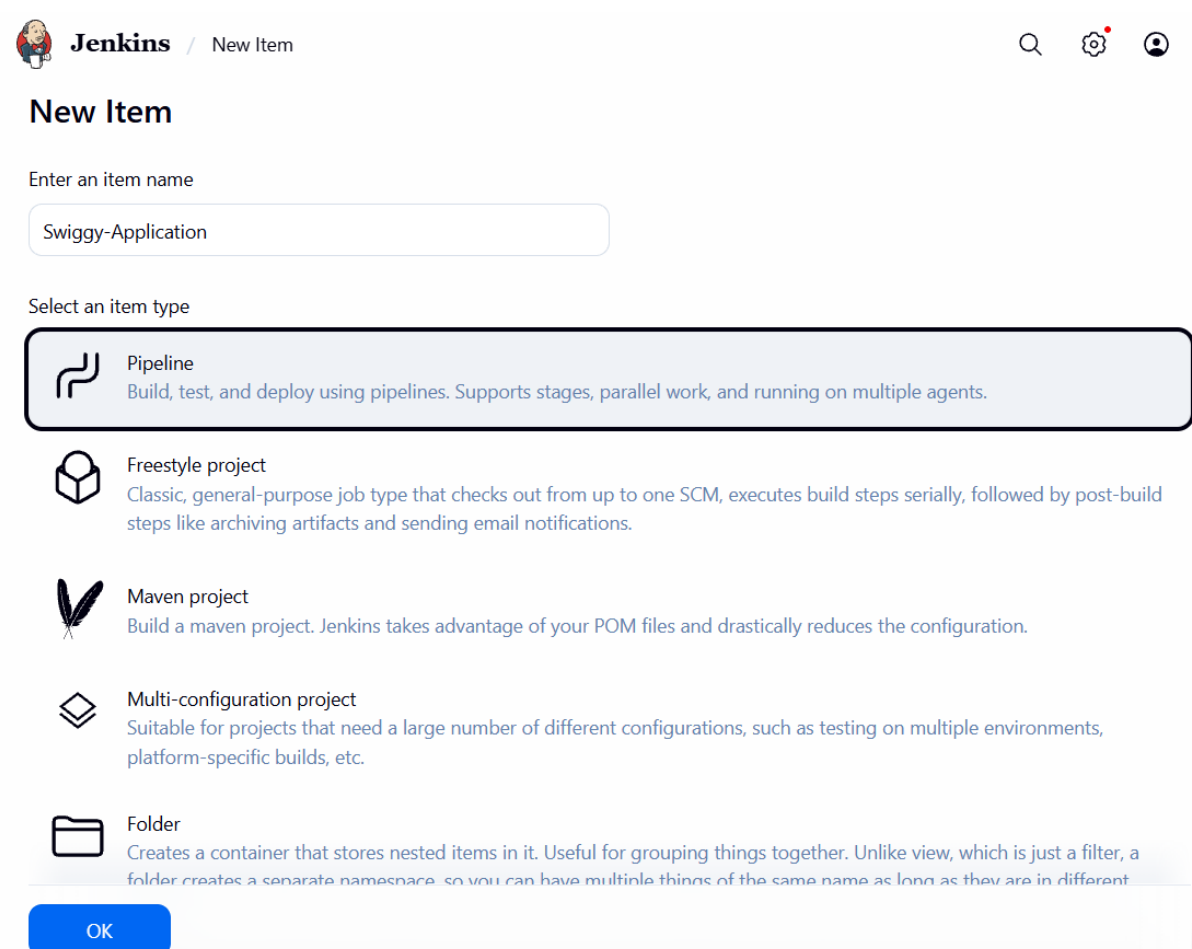
Save

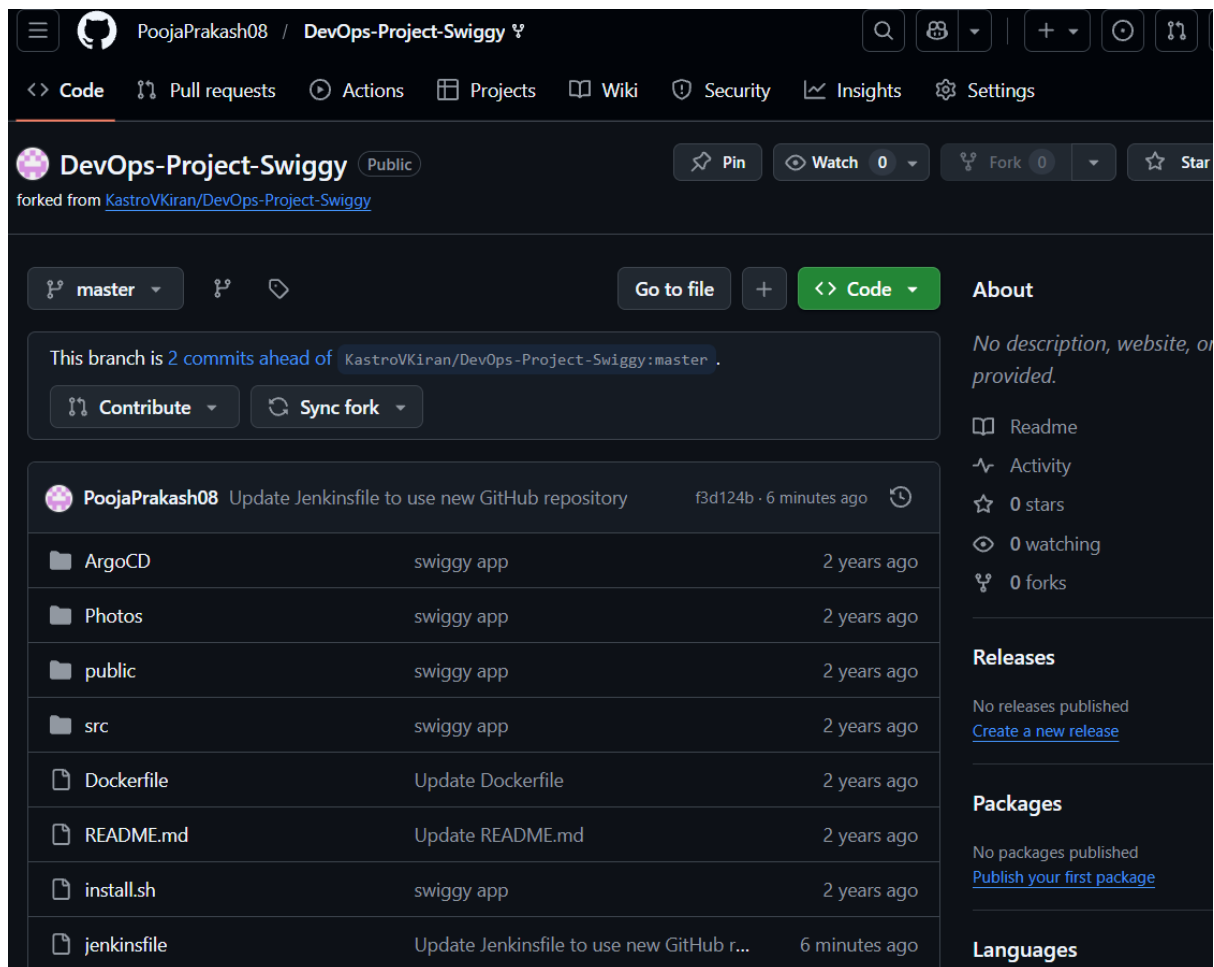
Apply

- Docker Hub credentials are added in Jenkins to enable secure image push operations.



- A Jenkins Pipeline job is created to automate the CI/CD process for the Swiggy application.





- The Jenkins pipeline performs the following stages: cloning the repository from GitHub, performing SonarQube analysis, waiting for Quality Gate validation, installing dependencies, running OWASP Dependency Check for vulnerability scanning, executing Trivy filesystem scan, building the Docker image, tagging and pushing the image to Docker Hub, performing Trivy image scan, and deploying the application container.

```
Welcome provider.tf Main.tf jenkinsfile X resource.sh
DevOps-Project-Swiggy > jenkinsfile
1 pipeline{
2     agent any
3     tools{
4         jdk 'jdk17'
5         nodejs 'node25'
6     }
7     environment {
8         SCANNER_HOME=tool 'sonar-scanner'
9     }
10    stages {
11        stage('clean workspace'){
12            steps{
13                cleanWs()
14            }
15        }
16        stage('Checkout from Git'){
17            steps{
18                git 'https://github.com/PoojaPrakash08/DevOps-Project-Swiggy.git'
19            }
20        }
21        stage("Sonarqube Analysis "){
22            steps{
23                withSonarQubeEnv('sonar-server') {
24                    sh ''' $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=Swiggy \
25                        -Dsonar.projectKey=Swiggy '''
26                }
27            }
28        }
29        stage("quality gate"){
30            steps {
31                script {
32                    waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token'
33                }
34            }
35        }
36    }
37 }
```

```
DevOps-Project-Swiggy > jenkinsfile
1 pipeline{
10    stages {
36        stage('Install Dependencies') {
37            steps {
38                sh "npm install"
39            }
40        }
41        stage('OWASP FS SCAN') {
42            steps {
43                dependencyCheck additionalArguments: '--scan ./ --disableYarnAudit --disableNodeAudit', odcInstallation: 'DP-Check'
44                dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
45            }
46        }
47        stage('TRIVY FS SCAN') {
48            steps {
49                sh "trivy fs . > trivyfs.txt"
50            }
51        }
52        stage("Docker Build & Push"){
53            steps{
54                script{
55                    withDockerRegistry(credentialsId: 'docker-creds', toolName: 'docker'){
56                        sh "docker build -t swiggy ."
57                        sh "docker tag swiggy poojaprakash08/swiggy:latest "
58                        sh "docker push poojaprakash08/swiggy:latest "
59                    }
60                }
61            }
62        }
63    }
64 }
```

```

stage("TRIVY"){
  steps{
    sh "trivy image poojaprakash08/swiggy:latest > trivy.txt"
  }
}
stage('Deploy to container'){
  steps{
    sh 'docker run -d --name swiggy -p 3000:3000 poojaprakash08/swiggy:latest'
  }
}
}

```

DevOps-Project-Swiggy > Dockerfile

```

1  # Using Node.js 16 as the base image
2  FROM node:16
3
4  # Setting up the working directory
5  WORKDIR /app
6
7  # Copying the package.json and package-lock.json files to the working directory
8  COPY package*.json ./
9
10 # Installation of npm dependency
11 RUN npm install
12
13 # Copy the application code
14 COPY . .
15
16 # Building of the React app
17 RUN npm run build
18
19 # Expose port 3000 to access app
20 EXPOSE 3000
21
22 # Start your Node.js server
23 CMD ["npm", "start"]
24

```

Configure

- General
- Triggers
- Pipeline
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline{
2   agent any
3   tools{
4     jdk 'jdk17'
5     nodejs 'node25'
6   }
7   environment {
8     SCANNER_HOME=tool 'sonar-scanner'
9   }
10  stages {
11    stage('clean workspace'){
12      steps{
13        cleanWs()
14      }
15    }
16  }
```

☒ Use Groovy Sandbox ?

Save

Apply

Status

Swiggy-Application

Add description

- Changes
- Build Now
- Configure
- Delete Pipeline
- Full Stage View
- SonarQube
- Stages
- Rename
- Pipeline Syntax

Stage View

		Declarative: Tool Install	clean workspace	Checkout from Git	Sonarqube Analysis	quality gate	Install Dependencies	OWASP FS SCAN	TRIVY FS SCAN	Docker Build & Push	TRIVY	Deploy to container
Average stage times: (full run time: ~6min 16s)		188ms	341ms	1s	12s	370ms	10s	3min 13s	2s	1min 25s	1min 8s	530ms
#4	Feb 22 22:18 No Changes	188ms	341ms	1s	12s	370ms (paused for 597ms)	10s	3min 13s	2s	1min 25s	1min 8s	530ms

SonarQube Quality Gate

Swiggy **Passed**
server-side processing: **Success**



Latest Dependency-Check

Permalinks

Builds >

Filter

Today

#4 4:48 PM

- The Swiggy Clone application is successfully deployed inside a Docker container and made accessible through port 3000.

